

Home work 8

Daniel O. Olofin

April 12, 2025

Question 1

Read the documentation for `read_xlsx()`, specifically the portion for the `range =` argument. Use this argument to construct an alternative but equivalent implementation for importing the EMG and kinematics data.

```
library(readxl)

EMG_Kin <- read_xlsx("EMG and Kinematics of walking.xlsx",
  skip = 2,
  col_names = c("Time_Kin",
    "Hip",
    "Knee",
    "Ankle",
    "Time_EMG",
    "VastusLateralis",
    "Gastrocnemius",
    "TibialisAnterior"))

EMG_Kin
```

```
## # A tibble: 65,514 x 8
##   Time_Kin  Hip  Knee Ankle Time_EMG VastusLateralis Gastrocnemius
##   <dbl> <dbl> <dbl> <dbl>   <dbl>         <dbl>         <dbl>
## 1      0    22.1  2.71 -2.85 0          0          0
## 2    0.02    23.4  2.92 -4.37 0.000466    0          0
## 3    0.04    23.6  3.40 -5.30 0.000931    0          0
## 4    0.06    23.8  2.98 -4.33 0.0014      0          0
## 5    0.08    23.7  2.58 -3.46 0.00186     0          0
## 6    0.1     23.0  2.98 -3.70 0.00233     0          0
## 7    0.12    22.7  3.24 -3.56 0.00279     0          0
## 8    0.14    22.9  3.28 -4.37 0.00326     0          0
## 9    0.16    23.5  3.37 -4.18 0.00372     0          0
## 10   0.18    23.3  3.32 -4.53 0.00419     0          0
## # i 65,504 more rows
## # i 1 more variable: TibialisAnterior <dbl>
```

Question 2

Pick up where we left off with the STAT 1401 survey example. Clean the Minutes to hometown variable and calculate the average. If someone entered a range, use the midpoint. For now use the techniques shown in

the previous lecture; later we may learn some string manipulation methods to automate it further.

```
library(tidyverse)

survey <- read.csv("STAT 1401 Survey Data - 2014 - Sheet1.csv",
  col.names = c(
    "What's the last movie you saw in the theater?",
    "On a scale of 1 star to 5 stars, how would you rate that movie?",
    "What is your height in inches?",
    "How many inches long is your foot? If you don't have something for measuring hand",
    "What is your gender?",
    "What city and state were you born in? If you were born outside the US, please put",
    "Do you consider yourself a vegetarian or vegan?",
    "Have you ever been in a car accident when YOU were the driver?",
    "Which of the following desserts do you prefer most?",
    "Which choice best reflects your political leanings?",
    "Which choice best reflects the political leanings of your mother?",
    "How many siblings do you have?",
    "How long does it take to drive from your hometown to Statesboro? Report your answer in minutes",
    "Do you give your permission for the instructor to use this data in examples? Your",
    "What is your major?"
  ),
  check.names = FALSE)

View(survey)
```

```
library(dplyr)
library(stringr)

# Rename the long column for ease of use
survey <- survey |>
  rename(`Minutes to hometown` =
    `How long does it take to drive from your hometown to Statesboro? Report your answer in minutes`)

# Extract the 'Minutes to hometown' column as a vector
minutes_to_hometown <- survey %>%
  pull(`Minutes to hometown`)

# Simulated sample data (for illustration)
minutes_to_hometown_data <- list(
  5, "270-300 minutes", 94, 105, "Maybe 6000 minutes", 35, 180, 600, 0, 180,
  "120 minutes", "90 minutes", 45, 70, "120 minutes", "45-50", 270, "160min",
  240, 972, 45, 210, 270, 150, 210, 90, 240, 90, 180, 50, 60, "240 minutes",
  210, 65, "90 minutes", 240, "210 minutes", 75, 210, 210, 240
)

# Function to clean character entries and extract numerical midpoint
clean_and_extract_midpoint <- function(x) {
  if (is.numeric(x)) {
    return(x)
  } else if (is.character(x)) {
    # Extract all numeric values from the string
    extracted_values <- as.numeric(unlist(str_extract_all(x, "\\d+")))

    # If numbers were found, return their mean (useful for ranges)
  }
}
```

```

    if (length(extracted_values) > 0) {
      return(mean(extracted_values))
    } else {
      return(NA)
    }
  } else {
    return(NA)
  }
}

# Apply the cleaning function to the list of responses
cleaned_data <- lapply(minutes_to_hometown_data, clean_and_extract_midpoint)

# Optionally convert to numeric vector
cleaned_data <- unlist(cleaned_data)

# View cleaned data
print(cleaned_data)

```

```

## [1] 5.0 285.0 94.0 105.0 6000.0 35.0 180.0 600.0 0.0 180.0
## [11] 120.0 90.0 45.0 70.0 120.0 47.5 270.0 160.0 240.0 972.0
## [21] 45.0 210.0 270.0 150.0 210.0 90.0 240.0 90.0 180.0 50.0
## [31] 60.0 240.0 210.0 65.0 90.0 240.0 210.0 75.0 210.0 210.0
## [41] 240.0

```

```

# Calculate average time to hometown (ignoring NA values)
average_time <- mean(cleaned_data, na.rm = TRUE)
cat("Average minutes to hometown:", average_time, "\n")

```

```

## Average minutes to hometown: 317.1585

```

```

#calculating the average
average_number_mins <- mean(unlist(cleaned_data), na.rm = TRUE)
cat("Average:", average_number_mins)

```

```

## Average: 317.1585

```

Question 3: Scrape a website.

(Grad version) Find a website and identify at least two variables to be scraped. These variables should not be in the same table. If they each happen to be in separate tables, extract them without using the `html_table()` function. The information may be on a single page or multiple pages. You should have at least two observations of each variable (so minimum ending data frame size is 2x2). In a polite manner, scrape these variables into a data frame and write it to a .csv file. You must do this from scratch rather than repeating a scraping tutorial.

```

library(rvest)
library(dplyr)
# URL for population data
url_population <- "https://en.wikipedia.org/wiki/List_of_countries_and_dependencies_by_population"

```

```

# URL for GDP data
url_gdp <- "https://en.wikipedia.org/wiki/List_of_countries_by_GDP_(nominal)"

# Read the HTML content
page_population <- read_html(url_population)

# Extract the table containing population data
table_population <- page_population %>% html_node("table.wikitable")

# Extract country names and populations
population_data <- table_population %>%
  html_table() %>%
  select(Country = 1, Population = 2)

# Clean the data
population_data <- population_data %>%
  mutate(Population = as.numeric(gsub(",", "", Population))) %>%
  filter(!is.na(Population))

# Read the HTML content
page_gdp <- read_html(url_gdp)

# Extract the table containing GDP data
table_gdp <- page_gdp %>% html_node("table.wikitable")

# Extract country names and GDPs
gdp_data <- table_gdp %>%
  html_table() %>%
  select(Country = 1, GDP = 2)

# Clean the data
gdp_data <- gdp_data %>%
  mutate(GDP = as.numeric(gsub(",", "", GDP))) %>%
  filter(!is.na(GDP))

# Merge datasets on Country
merged_data <- inner_join(population_data, gdp_data, by = "Country")

merged_data

```

```

## # A tibble: 188 x 3
##   Country      Population      GDP
##   <chr>         <dbl>     <dbl>
## 1 World       8137000000 115494312
## 2 India       1413324000  4270000
## 3 China       1408280000 19535000
## 4 United States 340110988 30338000
## 5 Indonesia    282477584  1493000
## 6 Pakistan     241499431   393100
## 7 Nigeria      223800000   194760
## 8 Brazil       212583750  2308000
## 9 Bangladesh   169828911   457700
## 10 Russia      146028325  2197000

```

```
## # i 178 more rows
```

```
write.csv(merged_data, "country_population_gdp.csv", row.names = FALSE)
```